



REMEAL PROJECT

Data Structure and Algorithms lab

Group Members:

Abdullah (09-131242-004)

Abdul Moiz (09-131242-002)

Muhammad Zohaib Saqib (09-131242-072)

Table of Contents

1. Repository	2
1. Introduction	2
2. Problem Statement	2
3. Objectives	2
4. Technologies Used	3
5. Data Structures Used	4
6. Algorithms Used	6
7. System Workflow	7
8. Modules Description	7
9. File Handling & Data Storage	8
10. UML & System Diagrams	9
11. User Interface Description	13
12. Testing	14
13. Challenges Faced	15
14. Future Enhancements	16
15. Conclusion	17
References	18

1. Github Repository:

<https://github.com/AbdullahOO4/DSA-Project->

2. Introduction

2.1 Background

Approximately 1.3 billion tonnes of food is wasted globally every year (FAO, 2019), while hundreds of millions go hungry. In Pakistan, food waste at events like weddings and festivals is especially high. ReMeal addresses this problem by providing a digital platform where surplus food is redirected from those who have it to those who need it, reducing both hunger and the environmental impact of food decomposition.

2.2 Purpose

ReMeal was built as a university-level DSA semester project with two goals: (1) demonstrate practical application of data structures in a real system, and (2) create a socially useful food redistribution platform. The system covers the full lifecycle of a donation — listing, request, donor approval, and rider delivery — implemented entirely in C++ with file-based persistent storage.

3. Problem Statement

Surplus food cannot reach people in need efficiently due to the lack of a coordinated digital platform. Specific sub-problems: donors have no easy way to list leftover food; receivers have no structured request system; there is no delivery coordination; no records exist to track history; and existing platforms require expensive infrastructure. ReMeal solves all five through its role-based system, FIFO request queue, donor approval workflow, rider portal, and file-based storage.

4. Objectives

- Build a functional web platform for donors to list food and receivers to request it
- Implement donor-controlled request approval with auto-generated delivery records
- Create a rider portal for claiming and tracking deliveries step-by-step
- Apply Linked Lists, Stack, Queue, BST, Hash Table, and Graph in the C++ backend
- Use File Handling for persistent, lightweight, database-free data storage
- Write clean, well-documented code suitable as a learning reference for students

5. Technologies Used

Technology	Role	How Used in ReMeal
HTML	Frontend Structure	5 pages: index, login, donor, receiver, rider dashboards

Technology	Role	How Used in ReMeal
CSS	Styling	style.css for clean, responsive layout across all pages
JavaScript	Frontend Logic	fetch() API calls, session memory, dynamic table rendering
Node.js	Build Tooling	Utility scripts and tooling used during development
Express.js	API Pattern Reference	API design follows Express.js patterns for future migration
C++ (Winsock2)	Backend Server	Custom HTTP server, all API endpoints, DSA logic
File Handling	Data Storage	5 text files: users, donations, requests, deliveries, logs

6. Data Structures Used

6.1 Linked List

Three linked lists store User, Donation, and Request nodes dynamically, growing as records are added. Traversal is used to filter and return records as JSON to the frontend.

6.2 Stack (LogStack – LIFO)

Records all system events in reverse-chronological order. Every action (register, donate, approve) is pushed to the top. Entries are also written to log.txt for persistent history.

6.3 Queue (ReqQueue – FIFO)

Holds pending food requests in submission order. The first receiver to request a donation gets the first chance at approval, ensuring fairness among competing requests.

6.4 Binary Search Tree (DonBST)

Donations are inserted sorted by food item name. In-order traversal returns them alphabetically — used to display a sorted listing on the receiver dashboard.

6.5 Hash Table (UserHT)

User records are stored keyed by email with $O(1)$ average lookup. Login checks credentials in constant time regardless of how many users are registered.

6.6 Graph (Adjacency List)

Models donor-receiver relationships. Each approved request adds a directed edge from donor to receiver, enabling BFS traversal to map the food-sharing network.

7. Algorithms Used

Algorithm	Complexity	Usage in ReMeal
Bubble Sort	$O(n^2)$	Sorts donation list alphabetically by food item name
Merge Sort	$O(n \log n)$	Efficient alternative sort for larger donation datasets
Binary Search (BST)	$O(\log n)$	Fast lookup of donations by food item via tree structure
In-Order Traversal	$O(n)$	Extracts sorted donations from BST for receiver display

Algorithm	Complexity	Usage in ReMeal
FIFO Queue Processing	$O(1)$	Processes pending requests in order of submission
BFS Graph Traversal	$O(V+E)$	Maps donor-to-receiver food sharing connections

8. System Workflow

1. User visits `http://localhost:8080` → C++ server serves `index.html` with live stats from `/getStats`
2. New user registers (`POST /register`) → added to `UserLinkedList`, `UserHT`, and `users.txt`
3. Login (`POST /login`) → `UserHT` $O(1)$ lookup → redirected to role-specific dashboard
4. Donor adds food (`POST /addDonation`) → inserted into `DonationLinkedList` and `DonBST` → saved to `donations.txt`
5. Receiver browses (`GET /getDonations`) → BST in-order returns alphabetically sorted available donations
6. Receiver requests (`POST /makeRequest`) → enqueued in `ReqQueue` → saved to `requests.txt`
7. Donor approves (`POST /updateRequest`) → status updated → Delivery auto-created → Graph edge added
8. Rider claims (`POST /claimDelivery`) → marks `in_transit` → marks delivered

9. Modules Description

9.1 Donor Module (donor.html)

Allows donors to add food donations (foodItem, qty, expiry, address), view their listings with status tracking, and manage incoming receiver requests with one-click Approve/Reject. Approval auto-creates a delivery record for riders.

9.2 Receiver Module (receiver.html)

Allows receivers to browse all Available donations, send formal requests, view status (Pending/Approved/Rejected), and cancel any pending request before it is processed.

9.3 Rider Module (rider.html)

Shows unclaimed approved deliveries. Riders claim a delivery, update it to In Transit after pickup, then mark it Delivered on drop-off. Each entry shows donor and receiver pickup/drop-off addresses.

8.4 C++ Backend Server

Custom HTTP server on port 8080 using Winsock2. Parses raw HTTP requests, routes them to handlers, applies DSA-backed business logic, generates JSON responses, and handles all file I/O with mutex locking for thread safety.

10. File Handling & Data Storage

File	Fields (pipe-separated)
users.txt	id name email password phone role street area city vehicle vehicleNo

File	Fields (pipe-separated)
donations.txt	id donorID donorName foodItem qty expiry location status time
requests.txt	id receiverID receiverName donationID foodItem status time receiverAddress donorName
deliveries.txt	id requestID donorID donorName receiverID receiverName foodItem qty riderID riderStatus claimedAt
log.txt	Timestamped activity log entries (append-only, managed by LogStack)

On each API call the server opens the relevant file, reads records line-by-line (std::getline), splits by pipe separator, loads into the DSA structure, processes the request, then writes updated data back. A std::mutex ensures thread-safe concurrent access.

11. UML & System Diagrams

11.1 Use Case Diagram

Shows the three actors (Donor, Receiver, Rider) and all actions available to each in the ReMeal platform.

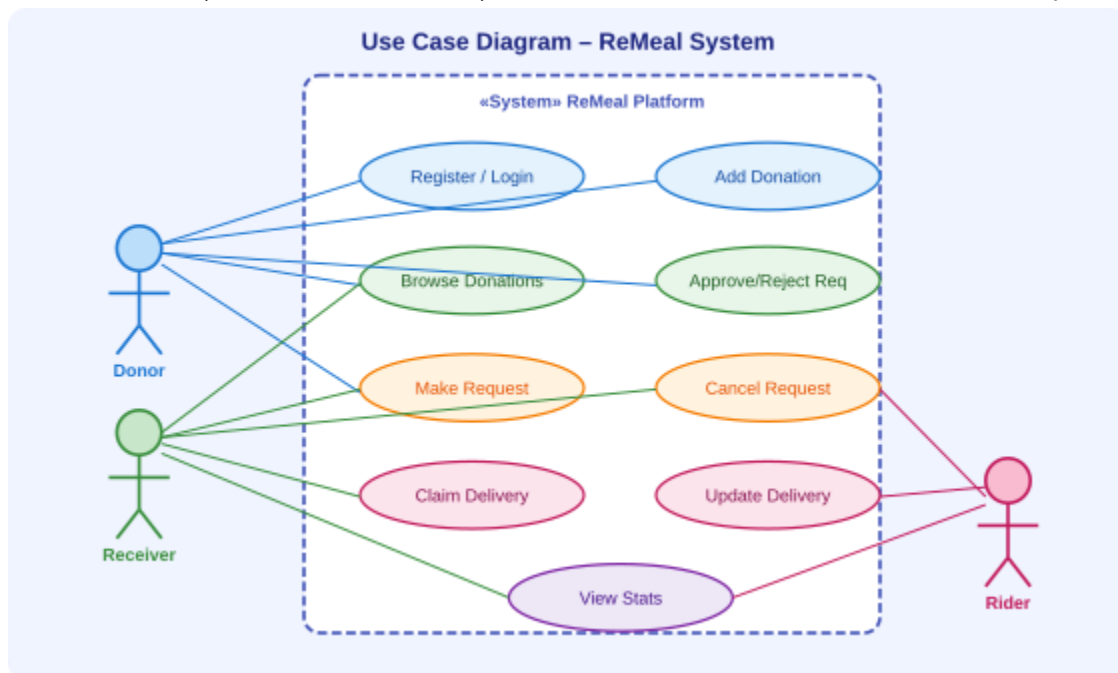


Figure 10.1 – Use Case Diagram for ReMeal System

11. Class Diagram

Shows C++ structs and DSA container classes with their attributes and key methods.

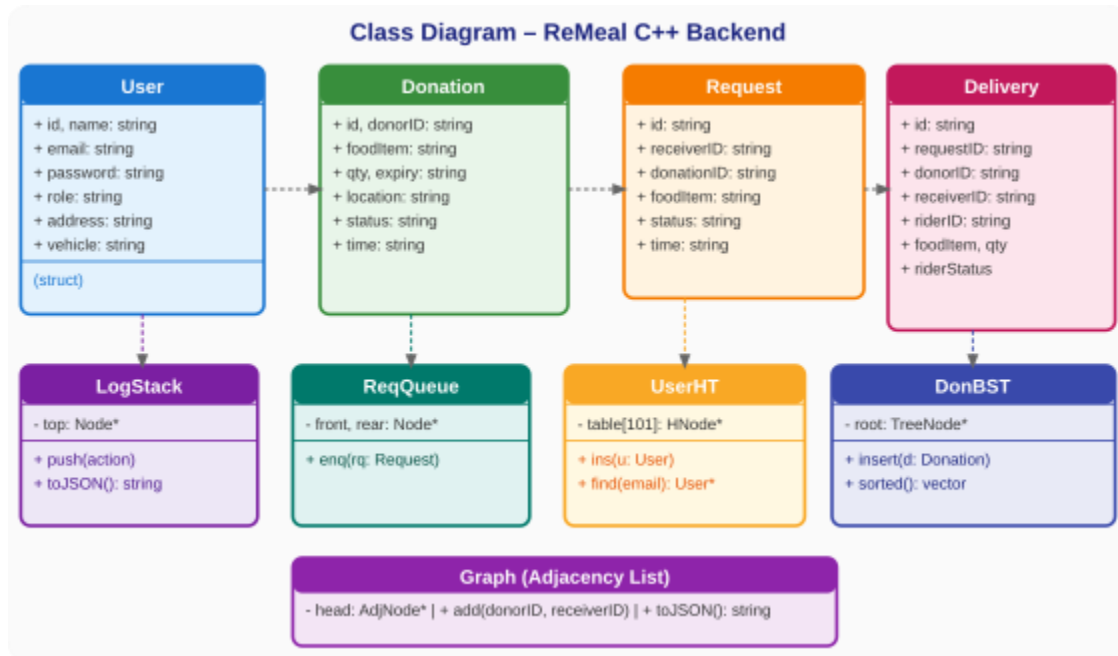


Figure 10.3 – Class Diagram for ReMeal C++ Backend

11.3 System Flowchart

Step-by-step decision flow from opening the app through registration, login, role selection, and donation processing.

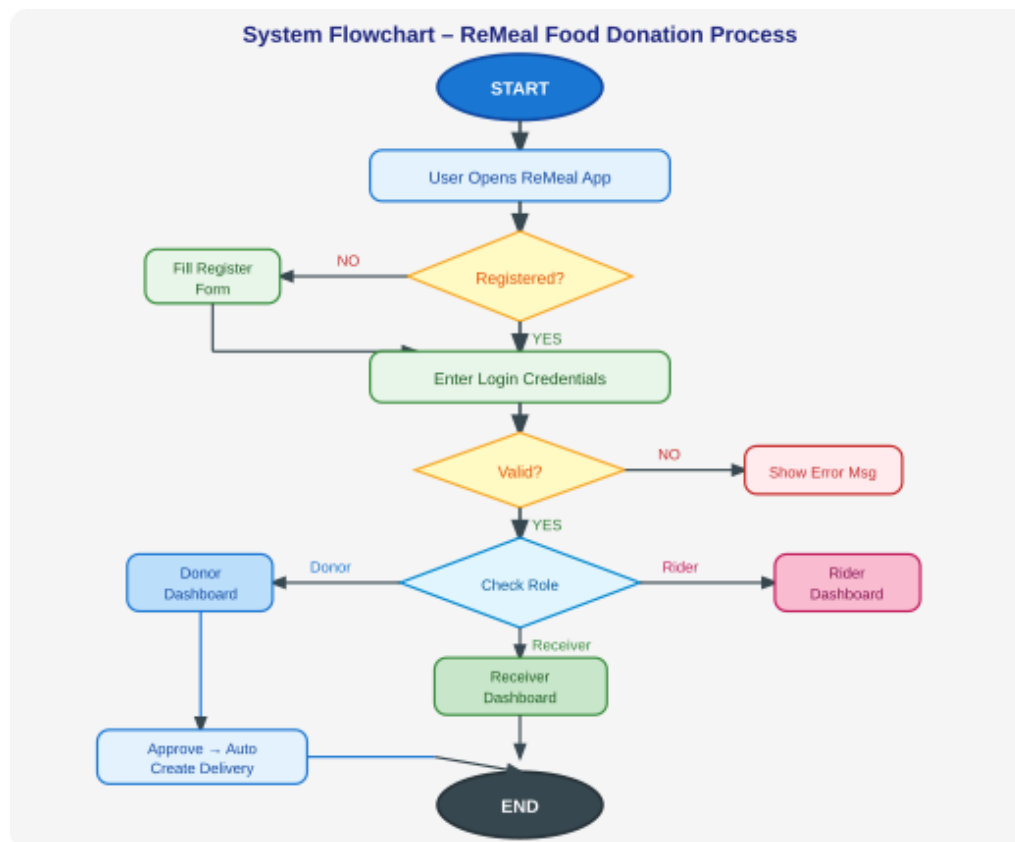


Figure 10.4 – System Flowchart for ReMeal

11.4 Sequence Diagram

Order of interactions between Receiver, Frontend, C++ Server, File Storage, and Donor during a complete request-and-approval cycle.

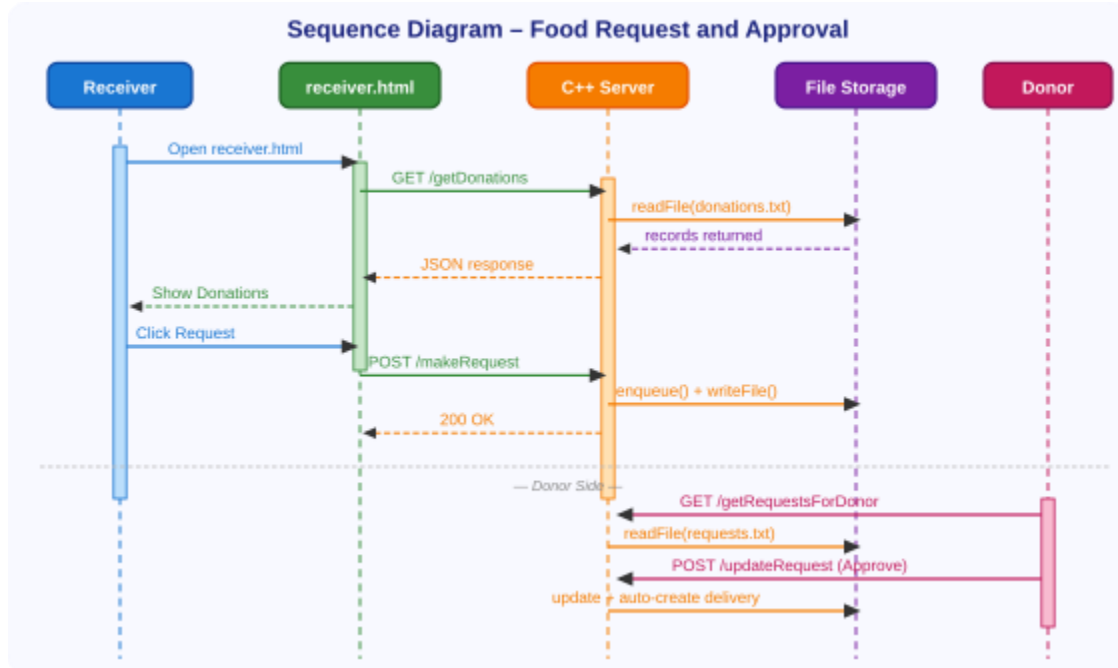


Figure 10.5 – Sequence Diagram for Request & Approval Flow

11.5 Activity Diagram

Complete delivery lifecycle across four swim lanes: Donor, Receiver, Rider, and System.

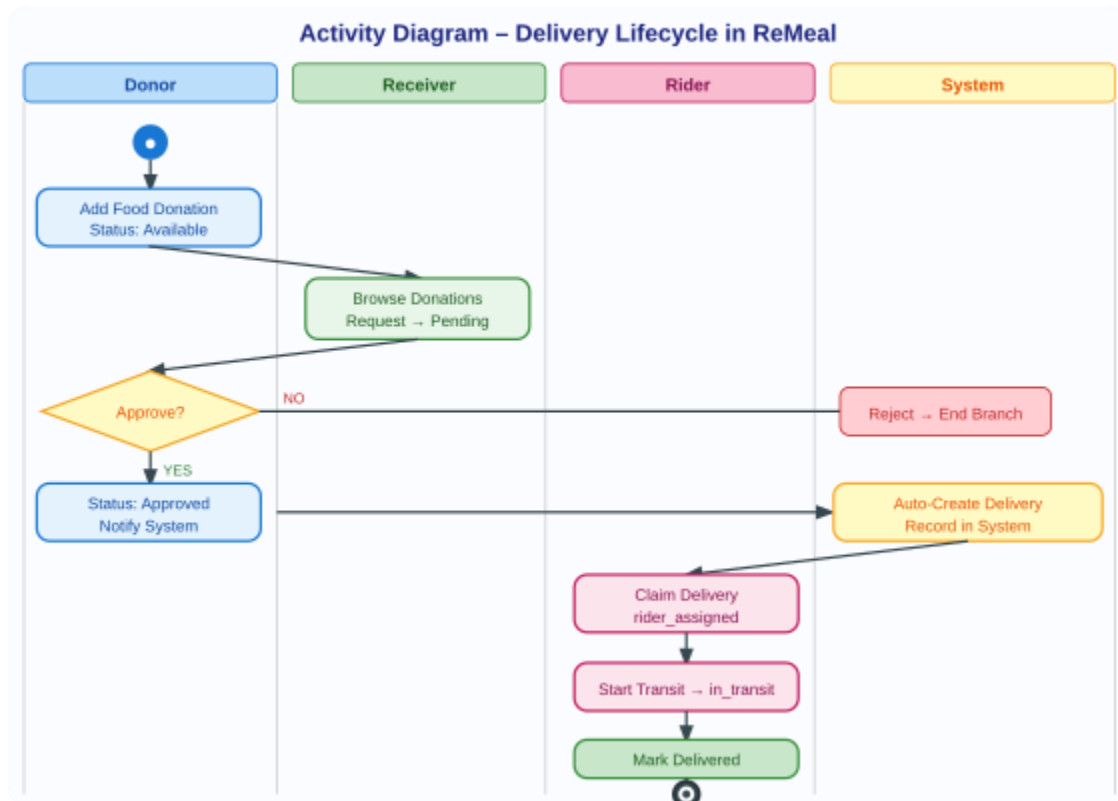


Figure 10.6 – Activity Diagram for Delivery Lifecycle

12. User Interface Description

Page	Key Elements & Functionality
index.html (Home)	Landing page: ReMeal tagline, live stats (donations/requests/deliveries), Get Started button
login.html	Two tabs: Login (email + password) and Register (name, email, password, phone, role, address, vehicle for riders)
donor.html	Three sections: Add Donation form My Donations table Incoming Requests with Approve/Reject buttons
receiver.html	Browse Available donations with Request button My Requests with status and Cancel option for Pending
rider.html	Available Deliveries with Claim button My Deliveries with Start Transit and Mark Delivered buttons

13. Testing

TC#	Test Case	Expected Result	Actual Result	Status	Notes
TC01	Register with valid data	Account created, redirect	Account created successfully	PASS	All roles
TC02	Register duplicate email	Error: Email already exists	Error shown correctly	PASS	Hash check
TC03	Login correct credentials	Dashboard shown	Dashboard loads correctly	PASS	O(1) lookup
TC04	Login wrong password	Error: Invalid credentials	Error message shown	PASS	
TC05	Donor adds donation	Listed as Available	Appears in receiver list	PASS	BST insert
TC06	Receiver views donations	All Available shown sorted	Table populated	PASS	In-order BST
TC07	Receiver makes request	Request: Pending status	Shows in My Requests	PASS	Queue enq
TC08	Receiver cancels request	Request removed	Deleted from list	PASS	
TC09	Donor approves request	Approved + delivery created	Delivery in rider list	PASS	Auto-create

TC#	Test Case	Expected Result	Actual Result	Status	Notes
TC10	Donor rejects request	Status = Rejected	Marked Rejected	PASS	
TC11	Rider claims delivery	Assigned to rider	In My Deliveries	PASS	
TC12	Rider marks delivered	Status = delivered	Marked complete	PASS	
TC13	Data persists after restart	Records reload from files	All data preserved	PASS	File I/O
TC14	Bubble Sort on donations	A-Z sort by food item	Correctly sorted	PASS	Algorithm
TC15	Concurrent write protection	No corruption with mutex	Files consistent	PASS	Thread-safe

14. Challenges Faced

- Building HTTP server from scratch in C++ — manually parsing raw HTTP strings, extracting method/path/body, and constructing valid JSON responses without any library.
- Thread safety with file locking — using `std::mutex` (`fileMutex`) to wrap every file read/write operation, preventing data corruption from concurrent requests.
- Keeping all DSA structures synchronized with file storage — every add/update must atomically update both the in-memory structure and the corresponding text file.
- Manual JSON generation in C++ — writing an `esc()` function to handle quotes, backslashes, and newlines correctly when building JSON strings character-by-character.
- CORS handling — adding `Access-Control-Allow-Origin: *` and correct `Content-Type` headers to all C++ server responses so browser `fetch()` calls are not blocked.

15. Future Enhancements

Enhancement	Description
Database Integration	Replace text files with MySQL or MongoDB for scalability, complex queries, and concurrent access
Mobile Application	Android/iOS app with push notifications for real-time donor-receiver-rider communication
AI Recommendations	ML model to match donors and receivers based on location, food type, and historical preferences
Location Services	Google Maps integration for proximity-based donation search and live GPS delivery tracking
Online Donations	Payment gateway (JazzCash/Stripe) for financial contributions to fund rider operations

Enhancement	Description
Real-time Notifications	WebSocket-based instant alerts when requests are submitted, approved, or deliveries updated
Admin Dashboard	System-wide panel for user moderation, analytics reports, and abuse management

16. Conclusion

ReMeal is a complete, functional food redistribution system that bridges the gap between food donors and those in need. It demonstrates that university-level DSA concepts — Linked Lists, Stacks, Queues, BSTs, Hash Tables, and Graphs — are powerful tools for building real software that solves real problems. The system achieves all its primary objectives: a working role-based web platform, a donor-controlled approval workflow, rider delivery coordination, and persistent file-based storage — all implemented with a custom C++ HTTP server built from first principles. With future enhancements like database integration, a mobile app, and AI-based food matching, ReMeal has genuine potential to grow into a production-ready platform that meaningfully reduces food waste and hunger in local communities.

References

9. Cormen, T. H. et al. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
10. Stroustrup, B. (2013). The C++ Programming Language (4th ed.). Addison-Wesley.
11. Microsoft Docs. (2023). Winsock2 Reference. https://learn.microsoft.com/en-us/windows/win32/api/_winsock/
12. Mozilla Developer Network. (2024). Fetch API. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
13. FAO. (2019). The State of Food and Agriculture – Food Loss and Waste Reduction. United Nations.
14. GeeksforGeeks. (2024). Data Structures Tutorial. <https://www.geeksforgeeks.org/data-structures/>
15. W3Schools. (2024). HTML, CSS, JavaScript Tutorials. <https://www.w3schools.com/>
16. MDN Web Docs. (2024). HTTP Overview. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview>

— End of Report —